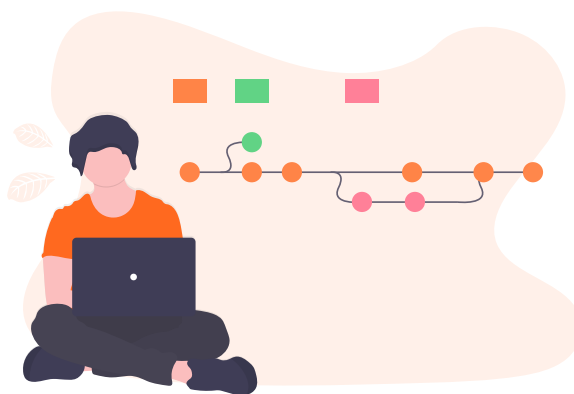


[Home](#) [About](#)[Login](#)

Java Versions and Features

Last updated on May 04, 2023 - [118 comments](#)

Quick Links

- [Practical Information](#)
- [Java Distributions](#)
- [Java Features 8-20](#)
- [Fin](#)
- [Acknowledgements](#)

↑ Top

You can use this guide to get practical information on how to find and install the latest Java, understand the differences between Java distributions (Adoptium, AdoptOpenJdk, Temurin, OpenJDK, OracleJDK etc.), as well as get an overview of Java language features, including version Java versions 8-19.

Practical Information

First, let's have a look at some common, practical questions that people have when trying to choose the right Java version for their project.

TL;DR I only want a download link and know about everything else. Where should I go?

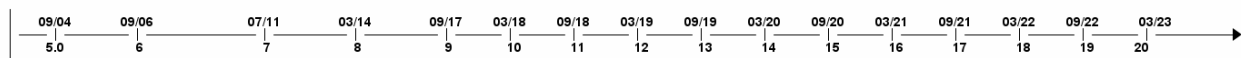
Go to the [Adoptium](#) site, choose the very latest release version, download and install it. Then come back to this guide to maybe still learn a thing or two about Java versions.

What is the latest Java version?

As of March 2023, [Java 20](#) is the latest released Java version. In September 2023, [Java 21](#) will follow. The current long-term support version (LTS) of Java is Java 17, released in September 2021.

What Java version should I use?

Newer Java versions now follow every 6 months. Hence, Java 21 is scheduled for September 2023, Java 22 for March 2024 and so on. In the past, Java release cycles were *much longer*, up to 3-5 years. This graphic demonstrates that:



With that many new versions coming out, there's basically these real-world[™] usage scenarios:

- Legacy projects in companies are often stuck with using Java 8 (see [Why are companies still stuck with Java 8?](#)). Hence you will be forced to use Java 8 as well.
- Some legacy projects are even stuck on Java 1.5 (released 2004) or 1.6 (released 2006) - sorry, folks, I feel for you!
- If you are making sure to use the very latest IDEs, frameworks and build tools and starting a greenfield project, you can, without hesitation, use Java 17 (LTS) or even the very latest Java 19.
- There's the special field of Android development, where the Java version is basically stuck at Java 7, with a [specific set of Java 8 features](#) available. Or you switch to using the Kotlin programming language.

Why are companies still stuck with Java 8?

There's a mix of different reasons some companies are still stuck with Java 8. To name a few:

- Build tools (Maven, Gradle etc.) and some libraries *initially* had bugs with versions Java versions > 8 and needed updates. For example, certain build tools like Maven would print out "reflective access"-warnings when building Java projects, which simply "felt not ready", even though the builds were fine.
- Up until Java 8 you were pretty much using Oracle's JDK builds and you did not have to care about licensing. Oracle changed the [licensing scheme In 2019](#), though, which led the internet go crazy with a ton of articles saying "Java is not free anymore" - and a fair amount of confusion followed. This is however not really an issue, which you'll learn about in the [Java Distributions](#) section of this guide.
- Some companies have policies to only use LTS versions and rely on their OS vendors to provide them these builds, which takes time.

To sum up: you have a mix of practical issues (upgrading your tools, libraries, frameworks) and political issues.

Why are some Java versions, like 8 also called 1.8?

Java versions before 9 simply had a different naming scheme. So, Java 8 can also be called *1.8*, Java 5 can be called *1.5* etc. When you issued the 'java -version' command, with these versions you got output like this:

```
c:\Program Files\Java\jdk1.8.0_191\bin>java -version
java version "1.8.0_191" (1)
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, mixed mode)
```

Which simply means Java 8. **With the switch to time-based releases with Java 9 the naming scheme also changed, and Java versions aren't prefixed with 1.x anymore.** Now the version number looks like this:

```
c:\Program Files\Java\jdk11\bin>java -version
openjdk version "11" 2018-09-25 (1)
OpenJDK Runtime Environment 18.9 (build 11+28)
OpenJDK 64-Bit Server VM 18.9 (build 11+28, mixed mode)
```

What is the difference between the Java versions? Should I learn a specific one?

Coming from other programming languages with major breakages between releases, like say Python 2 to 3, you might be wondering if the same applies to Java.

Java is special in this regard, as it is *extremely* backwards compatible. This means that your Java 5 or 8 program is guaranteed to run with a Java 8-20 virtual machine - with a few exceptions you don't need to worry about for now.

It obviously does not work the other way around, say your program relies on Java 20 features, that are simply not available under a Java 8 JVM.

This means a couple of things:

- You do not just "learn" a specific Java version, like 12.
- Rather, you'll get a good foundation in all language features up until Java 8. This serves as a good base.
- And then learn, from a guide like this, what *additional features* came in Java 9-20 and use them whenever you can.

What are examples of these new features between Java versions?

Have a look at the [Java Features 8-20](#) section.

But as a rule of thumb: The older, longer release-cycles (3-5 years, up until Java 8) meant a lot of new features per release.

The 6-month release cycle means a lot less features, per release, so you can catch up quickly on Java 9-20 language features.

What is the difference between a JRE and a JDK?

Up until now, we have only been talking about "Java". But what is Java exactly?

First, you need to differentiate between a *JRE* (Java Runtime Environment) and a *JDK* (Java Development Kit).

Historically, you downloaded just a *JRE* if you were only interested in running Java programs. A JRE includes, among other things, the Java Virtual Machine (JVM) and the "java" command line tool.

To develop new Java programs, you needed to download a *JDK*. A JDK includes *everything* the JRE has, as well as the compiler *javac* and a couple of other tools like *javadoc* (Java documentation generator) and *jdb* (Java Debugger).

Now why am I talking in past tense?

Up until Java 8, the Oracle website offered JREs and JDKs as separate downloads - even though the JDK also always included a JRE in a separate folder. With Java 9 that distinction was basically gone, and you are *always* downloading a JDK. The directory structure of JDKs also changed, with not having an explicit JRE folder anymore.

So, even though some distributions (see [Java Distributions](#) section) still offer a separate JRE download, there seems to be the trend of offering just a JDK. Hence, we are going to use *Java* and *JDK* interchangeably from now on.

How do I install Java or a JDK then?

Ignore the Java-Docker images, .msi wrappers or platform-specific packages for the moment. In the end, Java is just a .zip file, nothing more, nothing less.

Therefore, all you need to do to install Java onto your machine, is to unzip your jdk-{5-20}.zip file. You don't even need administrator rights for that.

Your unzipped Java file will look like this:

Directory C:\dev\jdk-11

12.11.2019	19:24	<DIR>	.
12.11.2019	19:24	<DIR>	..
12.11.2019	19:23	<DIR>	bin
12.11.2019	19:23	<DIR>	conf
12.11.2019	19:24	<DIR>	include

```

12.11.2019  19:24    <DIR>          jmods
22.08.2018  19:18    <DIR>          legal
12.11.2019  19:24    <DIR>          lib
12.11.2019  19:23                1.238 release

```

The magic happens in the `/bin` directory, which under Windows looks like this:

```

Directory C:\dev\jdk-11\bin
...
12.11.2019  19:23                272.736 java.exe
...
12.11.2019  19:23                20.832 javac.exe
...

```

So all you need to do is unzip that file and put the `/bin` directory in your `PATH` variable, so you can call the 'java' command from anywhere.

(In case you are wondering, GUI installers like the one from Oracle or Adoptium will do the unzipping and modifying the `PATH` variable for you, that's about it.)

To verify you installed Java correctly, you can then simply run 'java -version'. If the output looks like the one below, you are good to go.

```

openjdk version "11" 2018-09-25
OpenJDK Runtime Environment 18.9 (build 11+28)
OpenJDK 64-Bit Server VM 18.9 (build 11+28, mixed mode)

```

Now there's one question left: Where do you get that Java .zip file from? Which brings us to the topic of distributions.

Java Distributions

There's a variety of sites offering Java (read: JDK) downloads and it is unclear "who offers what and with which licensing". This section will shed some light on this.

The OpenJDK project

In terms of Java source code (read: the source code for your JRE/JDK), there is *only one*, living at the [OpenJDK project](#) site.

This is just source code however, not a distributable build (think: your .zip file with the compiled java command for your specific operating system). In theory, you and I could produce a build from that source code, call it, say, *MarcoJDK* and start distributing it. But our distribution would lack certification, to be able to legally call ourselves *Java SE compatible*.

That's why in practice, there's a handful of vendors that actually create these builds, get them certified (see [TCK](#)) and then distribute them.

And while vendors cannot, say, remove a method from the String class before producing a new Java build, they can add branding (yay!) or add some other (e.g. CLI) utilities they deem useful. But other than that, the original source code is *the same* for *all* Java distributions.

OpenJDK builds (by Oracle) and OracleJDK builds

One of the vendors who builds Java from source is Oracle. This leads to *two different Java distributions*, which can be very confusing at first.

1. [OpenJDK builds](#) by Oracle(!). These builds are free and unbranded, but Oracle won't release updates for older versions, say Java 15, as soon as Java 16 comes out.
2. [OracleJDK](#), which is a branded, commercial build starting with the license change in 2019. ~~Which means it can be used for free during development, but you need to pay Oracle if using it in production. For this, you get longer support, i.e. updates to versions and a telephone number you can call if your JVM goes crazy.~~ In September 2021, starting with Oracle Java 17, Oracle introduced the [No-Fee Terms and Conditions License](#), making OracleJDK free again, with a couple of limitations you can read about by spending hours on the Oracle website.

Now, historically (pre-Java 8) there were actual source differences between OpenJDK builds and OracleJDK builds, where you could say that OracleJDK was 'better'. But as of today, both versions are essentially the same, with [minor differences](#).

It then boils down to you wanting paid, commercial support (a telephone number) for your installed Java version.

Adoptium's Eclipse Temurin (formerly AdoptOpenJDK)

In 2017, a group of Java User Group members, developers and vendors (Amazon, Microsoft, Pivotal, Redhat and others) started a community, called [AdoptOpenJDK](#). As of August 2021, the AdoptOpenJDK project moved to a new home and is now called the [Eclipse Adoptium](#) project. [Here](#) is the current list of Adoptium Working Group Members.

Adoptium provides free, rock-solid OpenJDK builds, called *Eclipse Temurin*, with [longer availability/updates](#), across a variety of operating systems, architectures and versions.

Highly recommended if you are looking to install Java.

Azul Zulu, Amazon Corretto, SAPMachine

You will find a complete list of OpenJDK builds at the [OpenJDK Wikipedia](#) site. Among them are [Azul Zulu](#), [Amazon Corretto](#) as well as [SapMachine](#), to name a few. To oversimplify it boils down to you having different support options/maintenance guarantees.

Still, if you're, for example, working on AWS, it makes sense to just go with their Amazon Corretto OpenJDK builds, provided they offer the Java version you want to use.

A Complete OpenJDK Distribution Overview

[Rafael Winterhalter](#) compiled a great list of all available OpenJDK builds, including their OS, architecture, licensing, support and maintenance windows.

Check it out here: <https://rafael.codes/openjdk/>.

Recommendation

To re-iterate from the beginning, in 2023 and beyond, unless you have very specific requirements, go get your jdk.zip (.tar.gz/.msi/.pkg) file from <https://adoptium.net> (called *Eclipse Temurin*) or choose a package provided by your OS-vendor / cloud-provider.

Java Features 8-20

As mentioned at the very beginning of this guide: Essentially *all* (don't be picky now) Java 8 language features also work in Java 20. The same goes for all other Java versions in between.

Which in turns means that all language features from Java 8 serve as very good Java base knowledge and everything else (Java 9-20) is pretty much additional features *on top* of that baseline.

Here's a quick overview of what the specific versions have to offer:

- Java 8 -

Java 8 was a massive release and you can find a list of all features at [the Oracle website](#). There's two main feature sets I'd like to mention here, though:

Language Features: Lambdas etc.

Before Java 8, whenever you wanted to instantiate, for example, a new Runnable, you had to write an anonymous inner class like so:

```
Runnable runnable = new Runnable(){
    @Override
    public void run(){
        System.out.println("Hello world !");
    }
};
```

With lambdas, the same code looks like this:

```
Runnable runnable = () -> System.out.println("Hello world two!");
```

You also got method references, repeating annotations, default methods for interfaces and a few other language features.

Collections & Streams

In Java 8 you also got functional-style operations for collections, also known as the Stream API. A quick example:

```
List<String> list = Arrays.asList("franz", "ferdinand", "fiel", "vom", "pfe.
```

Now pre-Java 8 you basically had to write for-loops to do something with that list.

With the Streams API, you can do the following:

```
list.stream()  
    .filter(name -> name.startsWith("f"))  
    .map(String::toUpperCase)  
    .sorted()  
    .forEach(System.out::println);
```

If you want more Java 8 practice

Obviously, I can only give a quick overview of each newly added Stream, Lambda or Optional method in Java 8 in the scope of this guide.

If you want a more detailed, thorough overview - including exercises - you can have a look at my [Java 8 core features](#) course.

- Java 9 -

Java 9 also was a fairly big release, with a couple of additions:

Collections

Collections got a couple of new helper methods, to easily construct Lists, Sets and Maps.

```
List<String> list = List.of("one", "two", "three");  
Set<String> set = Set.of("one", "two", "three");  
Map<String, String> map = Map.of("foo", "one", "bar", "two");
```

Streams

Streams got a couple of additions, in the form of `takeWhile`, `dropWhile`, `iterate` methods.

```
Stream<String> stream = Stream.iterate("", s -> s + "s")
    .takeWhile(s -> s.length() < 10);
```

Optionals

Optionals got the sorely missed `ifPresentOrElse` method.

```
user.ifPresentOrElse(this::displayAccount, this::displayLogin);
```

Interfaces

Interfaces got private methods:

```
public interface MyInterface {

    private static void myPrivateMethod(){
        System.out.println("Yay, I am private!");
    }
}
```

Other Language Features

And a couple of other improvements, like an improved `try-with-resources` statement or diamond operator extensions.

JShell

Finally, Java got a shell where you can try out simple commands and get immediate results.

```
% jshell
| Welcome to JShell -- Version 9
| For an introduction type: /help intro

jshell> int x = 10
x ==> 10
```

HttpClient

Java 9 brought the initial preview version of a new HttpClient. Up until then, Java's built-in Http support was rather low-level, and you had to fall back on using third-party libraries like Apache HttpClient or OkHttp (which are great libraries, btw!).

With Java 9, Java got its own, modern client - although in preview mode, which means subject to change in later Java versions.

Project Jigsaw: Java Modules and Multi-Release Jar Files

Java 9 got the [Jigsaw Module System](#), which somewhat resembles the good old [OSGI specification](#). It is not in the scope of this guide to go into full detail on Jigsaw, but have a look at the previous links to learn more.

Multi-Release .jar files made it possible to have one .jar file which contains different classes for different JVM versions. So your program can behave differently/have different classes used when run on Java 8 vs. Java 10, for example.

If you want more Java 9 practice

Again, this is just a quick overview of Java 9 features and if you want more thorough explanations and exercises, have a look at the [Java 9 core features](#) course.

- Java 10 -

There have been a few changes to Java 10, like Garbage Collection etc. But the only real change you as a developer will likely see is the introduction of the "var"-keyword, also called local-variable type inference.

Local-Variable Type Inference: var-keyword

```
// Pre-Java 10
```

```
String myName = "Marco";
```

```
// With Java 10
```

```
var myName = "Marco"
```

Feels Javascript-y, doesn't it? It is still strongly typed, though, and only applies to variables *inside methods* (thanks, [dpash](#), for pointing that out again).

- Java 11 -

Java 11 was also a somewhat smaller release, from a developer perspective.

Strings & Files

Strings and Files got a couple new methods (not all listed here):

```
"Marco".isBlank();
```

```
"Mar\nc".lines();
```

```
"Marco ".strip();
```

```
Path path = Files.writeString(Files.createTempFile("helloWorld", ".txt"), "I  
String s = Files.readString(path);
```



Run Source Files

Starting with Java 10, you can run Java source files *without* having to compile them first. A step towards scripting.

```
ubuntu@DESKTOP-168M0IF:~$ java MyScript.java
```

Local-Variable Type Inference (var) for lambda parameters

The header says it all:

```
(var firstName, var lastName) -> firstName + lastName
```

HttpClient

The HttpClient from Java 9 in its final, non-preview version.

Other stuff

Flight Recorder, No-Op Garbage Collector, Nashorn-Javascript-Engine deprecated etc.

- Java 12 -

Java 12 got a couple [new features and clean-ups](#), but the only ones worth mentioning here are Unicode 11 support and a preview of the new switch expression, which you will see covered in the next section.

- Java 13 -

You can find a complete feature list [here](#), but essentially you are getting Unicode 12.1 support, as well as two new or improved preview features (subject to change in the future):

Switch Expression (Preview)

Switch expressions can now return a value. And you can use a lambda-style syntax for your expressions, without the fall-through/break issues:

Old switch statements looked like this:

```
switch(status) {  
    case SUBSCRIBER:  
        // code block  
        break;  
    case FREE_TRIAL:  
        // code block  
        break;  
    default:
```

```
// code block  
}
```

Whereas with Java 13, switch statements can look like this:

```
boolean result = switch (status) {  
    case SUBSCRIBER -> true;  
    case FREE_TRIAL -> false;  
    default -> throw new IllegalArgumentException("something is murky!");  
};
```

Multiline Strings (Preview)

You can *finally* do this in Java:

```
String htmlBeforeJava13 = "<html>\n" +  
    "    <body>\n" +  
    "        <p>Hello, world</p>\n" +  
    "    </body>\n" +  
    "</html>\n";
```

```
String htmlWithJava13 = ""  
    <html>  
        <body>  
            <p>Hello, world</p>  
        </body>  
    </html>  
    "";
```

- Java 14 -

Switch Expression (Standard)

The switch expressions that were *preview* in versions 12 and 13, are now standardized.

```
int numLetters = switch (day) {  
    case MONDAY, FRIDAY, SUNDAY -> 6;  
    case TUESDAY                 -> 7;  
    default                     -> {  
        String s = day.toString();  
        int result = s.length();  
        yield result;  
    }  
};
```

Records (Preview)

There are now record classes, which help alleviate the pain of writing a lot of boilerplate with Java.

Have a look at this pre Java 14 class, which only contains data, (potentially) getters/setters, equals/hashcode, toString.

```
final class Point {  
    public final int x;  
    public final int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}  
  
// state-based implementations of equals, hashCode, toString  
// nothing else
```

With records, it can now be written like this:

```
record Point(int x, int y) { }
```

Again, this is a preview feature and subject to change in future releases.

Helpful NullPointerExceptions

Finally NullPointerExceptions describe *exactly* which variable was null.

```
author.age = 35;
---
Exception in thread "main" java.lang.NullPointerException:
    Cannot assign field "age" because "author" is null
```

Pattern Matching For InstanceOf (Preview)

Whereas previously you had to (cast) your objects inside an instanceof like this:

```
if (obj instanceof String) {
    String s = (String) obj;
    // use s
}
```

You can now do this, effectively dropping the cast.

```
if (obj instanceof String s) {
    System.out.println(s.contains("hello"));
}
```

Packaging Tool (Incubator)

There's an incubating *jpackage* tool, which allows to package your Java application into platform-specific packages, including all necessary dependencies.

- Linux: deb and rpm
- macOS: pkg and dmg
- Windows: msi and exe

Garbage Collectors

The Concurrent Mark Sweep (CMS) Garbage Collector has been removed, and the experimental Z Garbage Collector has been added.

- Java 15 -

Text-Blocks / Multiline Strings

Introduced as an experimental feature in Java 13 (see above), multiline strings are now production-ready.

```
String text = """
    Lorem ipsum dolor sit amet, consectetur adipiscing \
    elit, sed do eiusmod tempor incididunt ut labore \
    et dolore magna aliqua.\
    """;
```

Sealed Classes - Preview

If you ever wanted to have an even closer grip on who is allowed to subclass your classes, there's now the *sealed* feature.

```
public abstract sealed class Shape
    permits Circle, Rectangle, Square {...}
```

This means that while the class is *public*, the only classes allowed to subclass *Shape* are *Circle*, *Rectangle* and *Square*.

Records & Pattern Matching

The *Records* and *Pattern Matching* features from Java 14 (see above), are still in preview and not yet finalized.

Nashorn JavaScript Engine

After having been deprecated in Java 11, the Nashorn Javascript Engine was now finally removed in JDK 15.

ZGC: Production Ready

The **Z Garbage Collector** is not marked experimental anymore. It's now production-ready.

- Java 16 -

Pattern Matching for instanceof

Instead of:

```
if (obj instanceof String) {  
    String s = (String) obj;  
    // e.g. s.substring(1)  
}
```

You can now do this:

```
if (obj instanceof String s) {  
    // Let pattern matching do the work!  
    // ... s.substring(1)  
}
```

Unix-Domain Socket Channels

You can now connect to Unix domain sockets (also supported by macOS and Windows (10+)).

```
socket.connect(UnixDomainSocketAddress.of(  
    "/var/run/postgresql/.s.PGSQL.5432"));
```

Foreign Linker API - Preview

A planned replacement for JNI (Java Native Interface), allowing you to bind to native libraries (think C).

Records & Pattern Matching

Both features are now production-ready, and not marked *in preview* anymore.

Sealed Classes

Sealed Classes (from Java 15, see above) are still in preview.

- Java 17 -

Java 17 is the new long-term support (LTS) release of Java, after Java 11.

Pattern Matching for switch (Preview)

Already available in many other languages:

```
public String test(Object obj) {  
  
    return switch(obj) {  
  
        case Integer i -> "An integer";  
  
        case String s -> "A string";  
  
        case Cat c -> "A Cat";  
  
        default -> "I don't know what it is";  
  
    };  
  
}
```

Now you can pass *Objects* to switch functions and check for a particular type.

Sealed Classes (Finalized)

A feature that was delivered in Java 15 as a preview is now finalized.

Recap: If you ever wanted to have an even closer grip on who is allowed to subclass your classes, there's now the *sealed* feature.

```
public abstract sealed class Shape
    permits Circle, Rectangle, Square {...}
```

This means that while the class is *public*, the only classes allowed to subclass *Shape* are *Circle*, *Rectangle* and *Square*.

Foreign Function & Memory API (Incubator)

A replacement for the Java Native Interface (JNI). Allows you to call native functions and access memory *outside* the JVM. Think C calls for now, but with plans for supporting additional languages (like C++, Fortran) over time.

Deprecating the Security Manager

Since Java 1.0, there had been a Security Manager. It's now deprecated and will be removed in a future version.

- Java 18 -

UTF-8 by Default

If you tried, e.g. reading in files without specifying an explicit character ending, the operating system encoding was used in previous Java versions (e.g. UTF-8 on Linux and macOS, and Windows-1252 on Windows). With Java 18 this changed to UTF-8 by default.

Simple Web Server

Java 18 now comes with a rudimentary HTTP server, that you can start with:

```
jwebserver
```

Learn more about its features [here](#).

Other Not-So-Exciting-Stuff / Incubating Features

For a full list and overview, check out [this article](#).

- Java 19 -

Java 19 added a couple of exciting features, like *Virtual Threads* and the new *Foreign Function & Memory API*, as well as *Structured Concurrency* and the *Vector API* - but they are all in preview mode, thus subject to change across the next releases.

If you want to read up on these features and what's to come, check out [this article](#).

- Java 20 -

Apart from some smaller features, Java 20 mainly iterated on previously introduced preview features: Scope Values, Record Patterns, Pattern Matching for switch, Foreign Function & Memory API, Virtual Threads & Structured Concurrency. All of them are work-in-progress, i.e. haven't been finalized yet.

If you want to read up on these features and what's to come, check out [this article](#).

Fin

By now you should have a pretty good overview of a couple things:

- How to install Java, which version to get and where to get it from (hint: [Adoptium](#)).
- What a Java distribution is, which ones exist and what the differences are.
- What the differences between the specific Java versions are.

Feedback, corrections and random input is always welcome! Simply leave a comment down below.

Thanks for reading.

Acknowledgements

Stephen Colebourne, who wrote a [fantastic article](#) on the different, available Java distributions.

There's more where that came from

I'll send you an update when I publish new guides. Absolutely no spam, ever. Unsubscribe anytime.



I'm not a robot

reCAPTCHA

Share



Comments

Login

Add a comment

M ↓ MARKDOWN



COMMENT ANONYMOUSLY

ADD COMMENT

Upvotes Newest Oldest

D

dulik

1 point · 3 years ago

Thanks for such a great article. In chapter "OpenJDK builds (by Oracle) and OracleJDK builds" chapter, you say "But as of today, both versions are essentially the same, with minor differences."

I was wondering about the differences, because the downloads at <https://jdk.java.net/> are so confusing - currently, there is the "Reference implementation" - <https://jdk.java.net/java-se-ri/15> and "Ready for use" <https://jdk.java.net/15/> - I've downloaded both, compared all the files by content and found they are 100% identical. So these 2 sources lead to the same OpenJDK 15 built by Oracle.

Then I downloaded "jdk-15_windows-x64_bin.zip" from Oracle Technology Network <https://www.oracle.com/java/technologies/javase-jdk15-downloads.html> - requires registration & login!). I compared the (Open)jdk-15 and (Oracle)jdk-15 directories by content and when ignoring dates, here are the results: /bin folder: 45 identical files, 79 files differing by their contents. No extra or missing files found. /lib folder: 12 identical, 8 different /jmods folder: 0 identical, 70 different etc.

The differences can be caused just by different text constants in the JDK source files, different compiler settings, but they are still there...

?

Anonymous**1 point** · 3 years ago

Thank you! I am learning and I wasn't aware that there was such significant changes from the 8 version to the 13.

T

typecalendar**0 points** · 4 months ago

Thank you for sharing this useful guide on finding and installing the latest version of Java. It's great to have practical information on different Java distributions and to get an overview of the language features in Java versions 8-19. This will definitely be a helpful resource for those looking to understand the differences between the various Java distributions and to stay up-to-date with the latest version of the programming language. I appreciate the time and effort you put into creating this guide. [february 2023](#)

?

Anonymous**1 point** · 3 years ago

Very helpful distinction of java versions (1.8 vs 13) and JRE vs JDK! Thank you for the article.

?

Anonymous**0 points** · 6 months ago

Thanks for the info.

?

Anonymous**0 points** · 6 months ago

Very useful info with attractive writing skills. good work

?

Anonymous**0 points** · 2 years ago

superb article! love from Malaysia

N

neeteshsoni7**0 points** · 2 years ago

Awesome information. Bookmarked it. Thank you a lot of tons.

M

m.lecaros.aranis**0 points** · 6 months ago

nice!

?

Anonymous**0 points** · 2 years ago

Best summary I've seen, thanks. I'll refer to it often and share it with my coworkers.

?

Anonymous**0 points** · 3 years ago

Really Useful Thanks

Y

yadavsachin0995**0 points** · 3 years ago

Thank you.

T

typecalendar**0 points** · 3 months ago

This means that while the class is public, the only classes allowed to subclass Shape are Circle, Rectangle and Square. [March 2023](#)

?

Anonymous**0 points** · 23 months ago

Awesome guide! I learned a lot. Thanks a lot for creating it.

?

Anonymous**0 points** · 21 months ago

Thank you! Great article!

?

Anonymous**0 points** · 2 years ago

Great Article. Very well written.

?

Anonymous**0 points** · 18 months ago

Thanks for the comprehensive - and up-to-date - content!

**Anonymous****0 points** · 3 years ago

Thanks a lot for this fantastic and informative article. Is there anything called AMD version of JDK and Intel version of JDK??

**Anonymous****0 points** · 15 months ago

Thank you were much for brief info.

**Anonymous****0 points** · 2 years ago

One of the best article I ever read. Extremely nice job. Thank's a lot.

**Anonymous****0 points** · 21 months ago

Thank you very much, I was out of the Java Universe for some time and your article was fantastic to resume the whole story!

**Anonymous****0 points** · 22 months ago

A very helpful, concise and an informative list of features and explanation. Thanks for the effort.

**Anonymous****0 points** · 20 months ago

Nice, thx! Little typo: "Java 18 is scheduled for March 2021"

**Anonymous****0 points** · 17 months ago

Thanks for the article!

**Anonymous****0 points** · 19 months ago

Exactly what I was looking for, thank you!

**Anonymous****0 points** · 18 months ago

I found this blog post by google search, I come from JS world and my new position is going to be in Java. You literally read my mind and answer tons of questions. Really kudos!

**Anonymous****0 points** · 16 months ago

Thank you for this overview. It's kind of hard to get elsewhere!

**giacomio.squicciarro****0 points** · 16 months ago

Great article!

**Anonymous****0 points** · 18 months ago

thanks

**Anonymous****0 points** · 18 months ago

Awesome article. Thanks for this wealth of information.

**Anonymous****0 points** · 13 months ago

Thank you very much for detailed information, you just hit the right spot with this guide! After 3 years pause in Java world this article sums up everything I need to know

**Anonymous****0 points** · 15 months ago

Hey man, huge thanks for this. Been coding Java for 25+, but i'm like a goldfish when it comes to stuff like this so this was a great help.

**Anonymous****0 points** · 14 months ago

Really clear and helpful article, thank you!

**Anonymous****0 points** · 15 months ago

Thanks for the highly helpful article. I'm using it to add the new language features to my introductory data structures course (CS2).

**Anonymous****0 points** · 14 months ago

Excellent article, much appreciated

**Anonymous****0 points** · 13 months ago

Thank you Marco!!!, Excellent article.

**Anonymous****0 points** · 24 months ago

I loved this article it was jus what i needed to decide

**Anonymous****0 points** · 2 years ago

Great job Marko, this is very informative, I really enjoyed reading it!

**Anonymous****0 points** · 21 months ago

Thank you! This article was very helpful.

**Anonymous****0 points** · 2 years ago

This article was very helpful, thank you so much. Just a heads up, Java 15 is out now! :)

**polovnikov.andrew****0 points** · 20 months ago

Great article, exatly what i've searched for! Could you please add java 17 features?

<https://openjdk.java.net/projects/jdk/17/>

**Anonymous****0 points** · 3 years ago

Thank you, thank you, thank you. Really well written and helps to clarify a lot.

**Anonymous****0 points** · 17 months ago

Thanks man, that was a good article.

**Anonymous****0 points** · 3 years ago

Thanks man!!!, I coded in java like 14 years ago and I was very lost with some stuff I am doing right now, thanks for the clear explanation

**Anonymous****0 points** · 14 months ago

Wonderful article. Thank you!

**Anonymous****0 points** · 17 months ago

Much helpful article Marco... Keep writing...

**Anonymous****0 points** · 2 years ago

Great Article. It increased my curiosity to learn more about Java instead of coding with existing known syntaxes. -
-Babu

**Anonymous****0 points** · 2 years ago

very good overview

**typecalendar****0 points** · 6 months ago

Since Java 1.0, there had been a Security Manager. It's now deprecated and will be removed in a future version.
Happy new year [2023](#)!

**Anonymous****0 points** · 5 months ago

Thanks a lot for this reference! It drives me nuts about the confusing JDK versions and domain names. Now all is clear!

**Anonymous****0 points** · 5 months ago

When a branch of a switch expression has a block of statements rather than a single expression, the yield statement inside that block terminates the evaluation of the switch 2023 expression, and the value in the yield statement becomes the value of the switch expression.

**Anonymous****0 points** · 5 months ago

Wow... this is really a good wake-up call for all the Java 8 tters.. Really appreciate your time in coming up with the overall structured details. Enjoyed reading!!

H

hdcaps.net**0 points** · 5 months ago

Awesome guide! I learned a lot. Thanks a lot for creating it. Also my website is:
<https://www.123calendars.com/2023-calendar.html>

?

Anonymous**0 points** · 4 months ago

Excellent content!

G

gratisonlinespil**0 points** · 5 months ago

A feature that was delivered in Java 15 as a preview is now finalized. [gratis spil](#)

?

Anonymous**0 points** · 4 months ago

Really Helpful!

?

Anonymous**0 points** · 4 months ago

great article

?

Anonymous**0 points** · 12 months ago

Thanks Marco for such a great article, it gave a detail insight of every version and addition of features alongwith it.

?

Anonymous**0 points** · 4 months ago

Thanks nice quick recap of all versions.

?

Anonymous**0 points** · 23 months ago

Thanks for the informations!

?

Anonymous**0 points** · 11 months ago

Perfect!

?

Anonymous**0 points** · 11 months ago

Thanks! for a quick overview of key features .

?

Anonymous**0 points** · 3 months ago

This is an absolute great summary of the topic. **Thank you for that :-)**

A

alex.velickiy**0 points** · 11 months ago

Thanks a lot. What a great article. Things got clear now.

?

Anonymous**0 points** · 11 months ago

Very much helpful to understand all required basic information at one place. Thanks a lot. !!

?

Anonymous**0 points** · 10 months ago

Absolutely wonderful article. You refueled my long forgone interest in analyzing technology growth (not just Java or Programming). Thank you Marco. - Nyx.

?

Anonymous**0 points** · 11 months ago

Wonderful reference, thanks a lot

?

Anonymous**0 points** · 11 months ago

So that's what 8ts all about! Thank you, very helpful .

H

harshay.imag3**0 points** · 10 months ago

Thank you for the article. It is a great recap of what has happened from Java 1.8 through now.

?

Anonymous**0 points** · 3 months ago

Very helpful as an android developer moving into the real Java world. Thank you for writing this!

Anonymous

?

0 points · 10 months ago

This is THE BEST summary I have come across. Many tech writeups are all but useless for those that need them and redundant for those that don't.

Was totally confused about Java 8, 1.8, etc until I read this. In particular the clarification of one source - openjdk too make things very clear. Thanks very much!

F

feb2023calendar**0 points** · 2 months ago

If you ever wanted to have an even closer grip on who is allowed to subclass your classes, there's now the sealed feature. [Calendar March 2023](#)

?

Anonymous**0 points** · 10 months ago

It's very good for me, thank you.

?

Anonymous**0 points** · 10 months ago

Thanks, Marco. I particularly appreciate the level of detail in the same features marked as both preview and production-ready, it allows for a much better comprehension of what's where.

?

Anonymous**0 points** · 2 months ago

I appreciate your effort to bring this content and explaining in detail. Easy to understand also. Thanks for sharing.

O

onur.yilmazer1**0 points** · 8 months ago

Thanks a lot. It would be a great convenience for the readers if you could add the year they came out in parentheses next to individual versions.

T

typecalendar**0 points** · 46 days ago

Recap: If you ever wanted to have an even closer grip on who is allowed to subclass your classes, there's now the sealed feature. [april 2023](#)

?

Anonymous**0 points** · 41 days ago

Great article, all in one place. I lost track of new Java features after Java8 - this helped me to grasp years of development in just a few minutes. Keep up the good work!

K

king**0 points** · 38 days ago

thanks Stephen Colebourne, who wrote a fantastic article on the different, available Java distributions.

?

Anonymous**0 points** · 7 days ago

"Hence, Java 21 is scheduled for September 2023, Java 22 for Marco 2024 and so on." Is "Marco" in this sentence a joke or a typo? :-)

T

typecalendar**0 points** · 31 days ago

Both features are now production-ready, and not marked in preview anymore. [May 2023 Calendar](#)

K

king**0 points** · 29 days ago

<https://www.atten-store.com>

K

king**0 points** · 24 days ago

ok, it is good idea, <https://www.sebart-store.com>

<https://www.lanavape.net>

?

Anonymous**0 points** · 3 years ago

Really well written. Perfect level of detail.

?

Anonymous**0 points** · 8 months ago

Detail information. Bookmarked it. Thank you .

?

Anonymous**0 points** · 8 months ago

thank you for this wonderfull article, i learned a thinks about java also feature's versions

Anonymous

?

0 points · 7 months ago

Very helpful, thanks!

?

Anonymous**0 points** · 22 months ago

Dear Marco. This is one of the best comprehensive articles I found in the past months helping me as a system integrator who wants to get rid of the license costs for Oracle Java to give stakeholders solid answers to their questions. Please keep it up-to-date as you do until now!

A

aleflerdev**0 points** · 3 years ago

Wow. I'm new to this world and starting BCs next week. The post you gave us is absolutely great. The fog of learning a programming was lifted a little more thank you.

J

jomosoto**0 points** · 22 months ago

Thanks for the article. It's a really great overview. But why there's still java 8 jre as standard download on Java.com?

J

jeremy.cox.cs**0 points** · 2 years ago

Thank you for a very helpful summary. I am curious about the idea of a "preview feature." I'd like to better understand. Releasing feature with unfinished syntax just seems like a way to waste my time. I don't get it.

?

Anonymous**0 points** · 2 years ago

Exquisite! A perfect level of detail. Thank you.

W

wifi2020123**0 points** · 2 years ago

thank you very much i like your article, it can help me

<http://www.arrows-hobby.net> <http://www.atten-store.com> <http://www.fms-model.com>
<http://www.keruistore.com> <http://www.freewing-model.com> <http://www.hobbystar.net>

M

me.mayukhchatterjee**0 points** · 21 months ago

This is such a well described article, lots of coverage done in such an efficient way, kudos!

**Anonymous****0 points** · 2 years ago

Wow, fantastic article, bookmarked for future reference. Content like this is what makes the internet go round.

**Anonymous****0 points** · 3 years ago

Hi Marco

Great article! I'd love to know why there are multiple versions of JDK builds - for example if you dig through <https://hub.docker.com/r/adoptopenjdk/openjdk14> you will see builds for both 14.0.1 and 14+36. Can you explain what the difference is between these two?

Gratefully yours J

**Marco Behler****0 points** · 3 years ago

Hi J, these are minor versions (14.0.1), including the build number (+36). These usually contain smaller bugfixes.

The Oracle homepage lists all those bug fixes and version differences quite nicely, have a look here:

JDK14: <https://www.oracle.com/technetwork/java/javase/14u-relnotes-6361871.html>

JDK11: <https://www.oracle.com/technetwork/java/javase/11u-relnotes-5093844.html>

**Anonymous****0 points** · 21 months ago

Thank you for your article, summarizes things clearly and nicely, But anything about Java 17? I think the EA version is out already, I'm planning to jump to 17 as soon as it is available.

**Anonymous****0 points** · 20 months ago

Great post. explained the concept very well.

**alejandrogomez****0 points** · 21 months ago

great article Marco, thanks!

**dmitrushchenkov****0 points** · 2 years ago

Please explain what is the 'yield result' in the java 14 switch?

**Anonymous****0 points** · 2 years ago

When a branch of a switch expression has a block of statements rather than a single expression, the yield statement inside that block terminates the evaluation of the switch expression, and the value in the yield statement becomes the value of the switch expression.

**Anonymous****0 points** · 2 years ago

Thank you! Very useful!

**Anonymous****0 points** · 2 years ago

It was an awesome read very comprehensive Thanks Much!!

**Anonymous****0 points** · 3 years ago

Well written article. Worth saving as a reference.

Mit freundlichen Grüßen Ingmar Pforr

**Anonymous****0 points** · 2 years ago

Excellent article. Thanks a ton!

Powered by **Commento**[Privacy & Terms](#) [Imprint](#) [Contact](#)

© 2021 Marco Behler GmbH